

Lab Assignment

C17 — STL `list<T>` — Core Operations

Define • Populate • Traverse • Search • Insert • Erase • Navigation • Splice

1. Objective

This lab provides hands-on practice with the **core operations of the STL `list<T>` container** — a doubly-linked list that supports efficient insertion and deletion at both ends and at any known position. Each activity isolates a specific group of methods so you can observe their behavior precisely.

By the end of this lab you will be able to:

- ▶ Define and populate a list using initializer lists, `push_back()`, `push_front()`, and `assign()`.
- ▶ Traverse a list using range-for, forward iterators, and reverse iterators.
- ▶ Search for elements using `find()` and `find_if()` from `<algorithm>`.
- ▶ Insert elements at any position using `insert()`.
- ▶ Remove elements safely using `erase()`, `pop_front()`, and `pop_back()`.
- ▶ Navigate with `advance()`, `next()`, `prev()`, and `distance()`.
- ▶ Move elements between lists using `splice()`.

2. Method Quick Reference

Method	Description
<code>l.push_back(val)</code>	Appends <code>val</code> to the end. $O(1)$.
<code>l.push_front(val)</code>	Prepends <code>val</code> to the front. $O(1)$.
<code>l.pop_back()</code>	Removes the last element. Returns void -- save <code>back()</code> first.
<code>l.pop_front()</code>	Removes the first element. Returns void -- save <code>front()</code> first.
<code>l.front()</code>	Returns reference to the first element.
<code>l.back()</code>	Returns reference to the last element.
<code>l.insert(pos, val)</code>	Inserts <code>val</code> BEFORE the iterator position <code>pos</code> .
<code>l.erase(pos)</code>	Removes element at <code>pos</code> . Returns iterator to next element.
<code>l.assign(n, val)</code>	Replaces all contents with <code>n</code> copies of <code>val</code> .
<code>l.assign({...})</code>	Replaces all contents with an initializer list.
<code>l.clear()</code>	Removes all elements. <code>size()</code> becomes 0.
<code>l.size()</code>	Returns the number of elements.
<code>l.empty()</code>	Returns true if <code>size() == 0</code> .
<code>l.splice(pos,src)</code>	Moves all elements of <code>src</code> into this list before <code>pos</code> . $O(1)$.
<code>advance(it, n)</code>	Moves iterator <code>it</code> by <code>n</code> steps in place (+ forward, - backward).
<code>next(it, n)</code>	Returns a new iterator <code>n</code> steps ahead. Original unchanged.
<code>prev(it, n)</code>	Returns a new iterator <code>n</code> steps behind. Original unchanged.
<code>distance(b, e)</code>	Returns number of steps between two iterators.

3. Shared Setup

Define the following helper function and `showList()` utility once at file scope. All seven activities use them.

```
#include <iostream>
#include <list>
#include <string>
#include <algorithm>    // find(), find_if()
#include <iterator>      // advance(), next(), prev(), distance()
using namespace std;

// Utility: print a list with an optional caption
template <class T>
void showList(const list<T>& lst, const string& caption = "") {
    cout << "\n" << caption << " (size=" << lst.size() << "):\n";
    for (const auto& item : lst)
        cout << " " << item << ", ";
    cout << endl;
}
```

4. Activities

Activity 1 — Define and Populate

Demonstrate three ways to create and fill a `list<string>`. Each method serves a different use case.

Tasks:

- ▶ Use an **initializer list** at construction to create `list1` with five Springfield characters.
- ▶ Create `list2` empty, then add elements one at a time using `push_back()` and `push_front()`.
- ▶ Create `list3` using `assign(4, "AVAIL")` to fill it with four copies of the same value.
- ▶ Print each list using `showList()` and confirm sizes.

```
void experiment01() {
    cout << "=== Activity 01: Define and Populate ===\n";

    // Method 1 -- initializer list
    list<string> list1 = { "Homer", "Marge", "Bart", "Lisa", "Maggie" };
    showList(list1, "list1 (initializer list)");

    // Method 2 -- push_back() and push_front()
    list<string> list2;
    list2.push_back("Lenny");
    list2.push_back("Carl");
    list2.push_front("Burns");    // inserts at the FRONT
    showList(list2, "list2 (push_back/push_front)");

    // Method 3 -- assign(count, value)
    list<string> list3;
    list3.assign(4, "AVAIL");
    showList(list3, "list3 (assign)");
}
```

Activity 2 — Traverse: Range-for, Forward Iterator, Reverse Iterator

Demonstrate all three traversal styles on the same list. A `list<T>` does not support operator[] -- iterators are the only way to walk the list.

Tasks:

- ▶ Traverse using a **range-for** loop -- elements arrive in order, head to tail.
- ▶ Traverse using a **forward iterator** from `begin()` to `end()`.
- ▶ Traverse in reverse using **`rbegin()`** and **`rend()`** -- tail to head.
- ▶ **Note:** set iterators are read-only -- all list iterators allow value modification via `it->` or `*it`.

```
void experiment02() {
    cout << "=== Activity 02: Traversal ===\n";
    list<string> lst = { "Homer", "Marge", "Bart", "Lisa", "Maggie" };

    // Style 1 -- range-for
    cout << "Range-for:          ";
    for (const auto& item : lst) cout << item << " ";
    cout << "\n";

    // Style 2 -- forward iterator
    cout << "Forward iterator:  ";
    for (auto it = lst.begin(); it != lst.end(); ++it)
        cout << *it << " ";
    cout << "\n";

    // Style 3 -- reverse iterator
    cout << "Reverse iterator:  ";
    for (auto it = lst.rbegin(); it != lst.rend(); ++it)
        cout << *it << " ";
    cout << "\n";
}
```

Activity 3 — Search: find() and find_if()

list<T> has no built-in search method. Use find() for exact matches and find_if() for condition-based searches. Both are in <algorithm> and return an iterator -- always check against end() before dereferencing.

Tasks:

- ▶ Use find() to locate "Bart" -- confirm with a result message.
- ▶ Use find() to search for "Krusty" -- confirm NOT FOUND.
- ▶ Use find_if() with a lambda to find the first name longer than 4 characters.
- ▶ Use find_if() with a capture [queryChar] to find the first name starting with 'M'.

```
void experiment03() {
    cout << "=== Activity 03: Search ===\n";
    list<string> lst = { "Homer", "Marge", "Bart", "Lisa", "Ned" };

    // find() -- exact match
    auto it = find(lst.begin(), lst.end(), "Bart");
    cout << "find(Bart): "
         << (it != lst.end() ? "FOUND --> " + *it : "NOT FOUND") << "\n";

    it = find(lst.begin(), lst.end(), "Krusty");
    cout << "find(Krusty): "
         << (it != lst.end() ? "FOUND" : "NOT FOUND (expected)") << "\n";

    // find_if() -- first name longer than 4 characters
    auto it2 = find_if(lst.begin(), lst.end(),
        [](const string& s) { return s.size() > 4; });
    if (it2 != lst.end())
        cout << "First name > 4 chars: " << *it2 << "\n";

    // find_if() -- first name starting with 'M' (capture)
    char queryChar = 'M';
    auto it3 = find_if(lst.begin(), lst.end(),
        [queryChar](const string& s) { return s[0] == queryChar; });
    if (it3 != lst.end())
        cout << "First name starting with 'M': " << *it3 << "\n";
}
```

Activity 4 — Insert at Any Position

`insert(pos, val)` places a new element BEFORE the iterator position `pos`. Use `find()` to locate the insertion point first.

Tasks:

- ▶ Start with `lst = { "Homer", "Bart", "Lisa" }` (Marge is missing).
- ▶ Use `insert()` to add "Marge" before "Bart".
- ▶ Use `insert(lst.begin(), val)` to add "Burns" at the front.
- ▶ Use `insert(lst.end(), val)` to add "Maggie" at the end (equivalent to `push_back`).
- ▶ Print the list after each insertion and confirm the size.

```
void experiment04() {
    cout << "=== Activity 04: Insert ===\n";
    list<string> lst = { "Homer", "Bart", "Lisa" };
    showList(lst, "Initial");

    // Insert "Marge" before "Bart"
    auto pos = find(lst.begin(), lst.end(), "Bart");
    lst.insert(pos, "Marge");
    showList(lst, "After insert(Marge before Bart)");

    // Insert at the front
    lst.insert(lst.begin(), "Burns");
    showList(lst, "After insert(Burns at front)");

    // Insert at the end (same effect as push_back)
    lst.insert(lst.end(), "Maggie");
    showList(lst, "After insert(Maggie at end)");
}
```

Activity 5 — Erase and Clear

`erase()` removes one element or a range and returns an iterator to the next surviving element. This return value must be captured when erasing inside a loop. `pop_front()` and `pop_back()` do not throw -- guard with `empty()` before calling them.

Tasks:

- ▶ Erase "Bart" by value: use `find()` then `erase(it)`.
- ▶ Erase the first element using `erase(lst.begin())`.
- ▶ Erase all names shorter than 5 characters using an iterator loop that captures the return value of `erase()`.
- ▶ Use `pop_front()` and `pop_back()` with `empty()` guards.
- ▶ Call `clear()` and verify `size()` is 0.

```
void experiment05() {
    cout << "=== Activity 05: Erase and Clear ===\n";
    list<string> lst = { "Burns", "Homer", "Marge", "Bart", "Lisa", "Ned" };
    showList(lst, "Initial");

    // Erase "Bart" by value
    auto it = find(lst.begin(), lst.end(), "Bart");
    if (it != lst.end()) lst.erase(it);
    showList(lst, "After erase(Bart)");

    // Erase first element
    lst.erase(lst.begin());
    showList(lst, "After erase(first)");

    // Erase all names shorter than 5 characters
    for (auto it2 = lst.begin(); it2 != lst.end(); ) {
        if (it2->size() < 5)
            it2 = lst.erase(it2);    // captures return value!
        else
            ++it2;
    }
    showList(lst, "After erase(length < 5)");

    // pop_front and pop_back with guards
    if (!lst.empty()) { cout << "pop_front: " << lst.front() << "\n"; lst.pop_front(); }
    if (!lst.empty()) { cout << "pop_back: " << lst.back() << "\n"; lst.pop_back(); }

    lst.clear();
    cout << "After clear(): size=" << lst.size() << "\n";
}
```

Activity 6 — Navigate: `advance()`, `next()`, `prev()`, `distance()`

`list<T>` does not support operator+ on iterators -- use `advance()` to move an iterator in place or `next()`/`prev()` to obtain a new iterator without modifying the original.

Tasks:

- ▶ Use `distance(begin, end)` to count elements between two positions.
- ▶ Use `advance(it, n)` to move forward (+n) and backward (-n).
- ▶ Use `next(it, n)` to peek n steps ahead without moving the original iterator.
- ▶ Use `prev(it, n)` to peek n steps behind without moving the original iterator.
- ▶ Confirm that the original iterator is unchanged after `next()` and `prev()`.

```
void experiment06() {
    cout << "=== Activity 06: Navigation ===\n";
    list<string> lst = { "Homer", "Marge", "Bart", "Lisa", "Maggie" };

    // distance() -- number of steps between two iterators
    int len = distance(lst.begin(), lst.end());
    cout << "distance(begin,end) = " << len << "\n";

    // advance() -- moves the iterator IN PLACE
    auto it = lst.begin();
    advance(it, 2);
    cout << "begin() + advance(2) ==> " << *it << "\n"; // Bart

    auto itEnd = lst.end();
    advance(itEnd, -2);
    cout << "end() + advance(-2) ==> " << *itEnd << "\n"; // Lisa

    // next() and prev() -- return NEW iterator, original unchanged
    auto itMid = next(lst.begin(), 2); // points to "Bart"
    cout << "next(begin, 2) ==> " << *itMid << "\n"; // Bart
    cout << "next(itMid, 1) ==> " << *next(itMid, 1) << "\n"; // Lisa
    cout << "prev(itMid, 1) ==> " << *prev(itMid, 1) << "\n"; // Marge
    cout << "itMid unchanged ==> " << *itMid << "\n"; // Bart
}
```

Activity 7 — Splice: Move Elements Between Lists

`splice()` moves elements from one list into another in $O(1)$ time -- no copying, no new nodes. The source list loses the moved elements. Three overloads are available.

Tasks:

- ▶ **Overload 1:** move the **entire** source list into the destination using `dst.splice(pos, src)`.
- ▶ **Overload 2:** move **one element** from source to destination using `dst.splice(pos, src, it)`.
- ▶ **Overload 3:** move a **range** of elements using `dst.splice(pos, src, first, last)`.
- ▶ After each splice confirm that the source list has **lost** the moved elements.

```
void experiment07() {
    cout << "=== Activity 07: Splice ===\n";

    list<string> plant = { "Homer", "Lenny", "Carl" };
    list<string> recruits = { "Frink", "Grimes", "Mindy" };

    // Overload 1 -- move entire recruits list before "Lenny"
    auto pos = find(plant.begin(), plant.end(), "Lenny");
    plant.splice(pos, recruits);
    showList(plant, "After splice(all recruits before Lenny)");
    showList(recruits, "recruits after (now empty)");

    // Overload 2 -- move one element from vips
    list<string> vips = { "Burns", "Smithers" };
    auto itBurns = find(vips.begin(), vips.end(), "Burns");
    plant.splice(plant.begin(), vips, itBurns);
    showList(plant, "After splice(Burns to front)");
    showList(vips, "vips after");

    // Overload 3 -- move a range from temps
    list<string> temps = { "Otto", "Barney", "Moe", "Ned" };
    auto first = find(temps.begin(), temps.end(), "Barney");
    auto last = find(temps.begin(), temps.end(), "Ned"); // not included
    plant.splice(plant.end(), temps, first, last);
    showList(plant, "After splice(Barney,Moe to end)");
    showList(temps, "temps after");
}
```

main() — Call All Activities

```
int main() {
    experiment01(); // Define and Populate
    experiment02(); // Traverse
    experiment03(); // Search
    experiment04(); // Insert
    experiment05(); // Erase and Clear
    experiment06(); // Navigate
    experiment07(); // Splice
    cout << "\nAll done!\n";
}
```

5. Self-Assessment: Quality Control

Check every item before submitting.

☐	Item
☐	Activity 01 -- three population methods produce correct lists and sizes.
☐	Activity 02 -- all three traversal styles produce correct head-to-tail and tail-to-head output.
☐	Activity 03 -- find() and find_if() results checked against end() before dereferencing.
☐	Activity 03 -- find_if() lambda uses correct capture syntax [queryChar] with return keyword.
☐	Activity 04 -- insert() places elements at middle, front, and end correctly.
☐	Activity 05 -- erase() return value captured in the iterator loop (it2 = lst.erase(it2)).
☐	Activity 05 -- pop_front() and pop_back() guarded with empty() before each call.
☐	Activity 06 -- advance() modifies iterator in place; next()/prev() leave original unchanged.
☐	Activity 07 -- source list loses elements after each splice (verified by printing both lists).
☐	showList() takes const list<T>& -- no unnecessary copies made.
☐	Code compiles with no errors and no warnings.
☐	All seven activities print labeled output that matches expected results.

Key concepts

- A doubly-linked list provides $O(1)$ insertion and deletion at any known position and at both ends.
- Unlike a vector it never needs to shift elements.
- The trade-off is that it has no random access -- reaching the n th element requires $O(n)$ pointer hops.
- The activities in this lab demonstrate both the strengths and the navigation techniques that compensate for that limitation.